



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

دانشکده‌ی مهندسی کامپیوتر

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین پنجم

فهرست مسائل

۱		مسئله ۱
۱		الف) External Network
۱		ب) Linting
۲		ج) Image Registry
۳		مسئله ۲
۵		مسئله ۳
۷		مسئله ۴

پاسخ مسئله‌ی ۱. تعریف مفاهیم

الف) External Network

وقتی با docker compose یک پروژه را بالا می‌آوریم، داکر به‌صورت پیش‌فرض برای همان پروژه یک شبکه‌ی اختصاصی می‌سازد و با پایین آوردن پروژه آن را حذف می‌کند. شبکه‌ی external یعنی شبکه‌ای که بیرون از این چرخه‌ی عمر ساخته شده است؛ ما فقط اعلام می‌کنیم که از قبل وجود دارد و داکر باید سرویس را به آن وصل کند، نه اینکه آن را بسازد یا پاک کند.

```

1 # docker network create network-name
2 networks:
3   network-name:
4     external: true

```

کاربرد اصلی‌اش این است که چند پروژه‌ی compose مستقل بتوانند همدیگر را با نام سرویس صدا بزنند، بدون اینکه مجبور باشیم همه را در یک فایل واحد جمع کنیم. نکته‌ی مهم این است که چون داکر کامپوز فعلی مالک آن شبکه نیست، اگر شبکه از قبل ساخته نشده باشد کامپوز خطا می‌دهد و پاک کردن آن هم وظیفه‌ی خود ماست.

ب) Linting

لینتینگ یعنی تحلیل ایستای کد؛ یعنی بررسی کد بدون اجرای آن برای پیدا کردن خطاهای احتمالی، الگوهای مشکوک و ناسازگاری با قواعد سبک تیم. تفاوتش با کامپایلر این است که کامپایلر فقط می‌پرسد آیا این کد معتبر است؟ ولی لینتر می‌پرسد آیا این کد خوب است؟: متغیر استفاده‌نشده، خطای نادیده‌گرفته‌شده، ایمپورت اضافی، نام‌گذاری خلاف قرارداد، پیچیدگی بیش از حد یک تابع و مواردی از این دست.

معمولاً دو لایه دارد: formatter که فقط ظاهر کد را یکدست می‌کند (مثل gofmt یا prettier) و linter که به معنا و ساختار کد کار دارد (مثل golangci-lint یا ESLint). ارزش اصلی‌اش در تیم این است که بحث‌های سلیقه‌ای از code review حذف می‌شود و بازبین به‌جای شمردن فاصله‌ها، روی منطق تمرکز می‌کند. به همین دلیل لینت تقریباً همیشه اولین و ارزان‌ترین مرحله‌ی پایپلاین است.

البته بعضاً مرحله فرمت و لینت را از هم جدا می‌کنند، اما بعضی تیم‌ها هم هر دو را در یک مرحله انجام می‌دهند.

ج) Image Registry

رجیستری، انبار نسخه‌بندی‌شده‌ی imageها است؛ همان نقشی که npm برای پکیج‌های جاوااسکریپت یا Maven Central برای جاوا دارد. ایمیج با نام و تگ در آن ذخیره می‌شود، با docker push بالا می‌رود و با docker pull پایین می‌آید.

نکته‌ی فنی مهم این است که رجیستری ایمیج را به‌صورت یک فایل یکپارچه نگه نمی‌دارد، بلکه یک manifest به‌علاوه‌ی لایه‌های ایمیج را ذخیره می‌کند و هر لایه با digest شناخته می‌شود؛ بنابراین لایه‌های مشترک بین ایمیج‌ها فقط یک بار ذخیره و منتقل می‌شوند. همین digest است که اجازه می‌دهد به‌جای تگ به image@sha256:... ارجاع دهیم.

پاسخ مسئله‌ی ۲. طراحی پایپلاین

در پروژه‌هایی که سرویس‌های مختلف شامل Frontend، Backend و تغییرات پایگاه داده در یک مخزن گیت مشترک نگهداری می‌شوند، طراحی پایپلاین CI/CD نیازمند جداسازی هوشمند مراحل، اجرای موازی برای بهینه‌سازی زمان، و ایجاد شرط‌های عبور محکم جهت کاهش ریسک استقرار است.

پایپلاین پیشنهادی شامل ۶ مرحله اصلی به ترتیب زیر است:

۱. Lint و تحلیل ایستا

• مراحل اجرا:

- اجرای لینترهای اختصاصی برای هر بخش (مانند ESLint برای Frontend و golangci-lint یا Flake8 برای Backend).
- بررسی کیفیت کد، قالب‌بندی، خطاهای نحوی و صحت نگارش Database Migration ها.
- شرط عبور: عدم وجود خطای Syntax یا شکست در قوانین تعریف‌شده (میزان Warnings قابل تنظیم است).
- خروجی: گزارش Lint و تحلیل کیفیت کد.

۲. تست

• مراحل اجرا:

- اجرای موازی Unit Tests برای Frontend و Backend.
- اجرای Integration Tests با استفاده از یک پایگاه داده ایزوله.
- بررسی صحت اجرای Database Migration ها و ارتباط صحیح Backend با پایگاه داده.
- شرط عبور: موفقیت تمامی تست‌های واحد و یکپارچه‌سازی و دستیابی به حداقل پوشش کد تعیین‌شده.
- خروجی: گزارش نتایج تست‌ها و گزارش پوشش کد.

۳. ساخت و انتشار ایمج

• مراحل اجرا:

- کامپایل پروژه‌ها و ساخت ایمج‌های داکر برای Frontend و Backend.
- برچسب‌گذاری ایمج‌ها بر اساس Commit SHA و نسخه.
- اسکن امنیتی ایمج‌ها برای شناسایی آسیب‌پذیری‌ها.
- Push ایمج‌ها به Container Registry.
- شرط عبور: ساخت موفق ایمج‌ها و عدم وجود آسیب‌پذیری بحرانی در اسکن امنیتی.
- خروجی: ایمج‌های داکر آماده استقرار در Registry.

۴. استقرار در محیط آزمایشی

• مراحل اجرا:

- اجرای خودکار Database Migration روی محیط Staging.
- به‌روزرسانی سرویس‌های Frontend و Backend.
- اجرای Smoke Test و تست‌های End-to-End.
- شرط عبور: استقرار موفق بدون Downtime و موفقیت کامل تست‌های دود.
- خروجی: محیط Staging به‌روز و آماده ارزیابی نهایی.

۵. تأیید دستی

• مراحل اجرا:

– بررسی نتایج تست‌ها، گزارش‌های امنیتی و وضعیت محیط Staging.

– تأیید انتشار توسط مسئول پروژه یا تیم عملیات.

• شرط عبور: دریافت تأییدیه انتشار.

• خروجی: مجوز آغاز استقرار در محیط عملیاتی.

۶. استقرار در محیط عملیاتی

• مراحل اجرا:

– اجرای Database Migration های سازگار با نسخه‌های قبلی.

– استقرار تدریجی با استفاده از Blue-Green Deployment یا Canary Deployment.

– بررسی Health Check ها و مانیتورینگ سرویس پس از انتشار.

– امکان بازگشت سریع در صورت مشاهده خطا.

• شرط عبور: سلامت کامل سرویس پس از استقرار و موفقیت تمامی Health Check ها.

• خروجی: نسخه جدید روی محیط عملیاتی به همراه مانیتورینگ و قابلیت بازگشت سریع.

تأیید دستی در دو نقطه کلیدی اضافه می‌شود:

۱. پیش از استقرار در محیط Production:

• دلیل: جلوگیری از انتشار ناخواسته کد به محیط اصلی، هماهنگی زمان انتشار با تیم‌های مارکتینگ/پشتیبانی، و انجام بررسی‌های نهایی توسط مدیر محصول یا لید فنی روی محیط Staging.

۲. پیش از اجرای Migration های حساس روی دیتابیس Production:

• دلیل: اسکرپت‌های تغییر ساختار دیتابیس ممکن است زمان‌بر یا سنگین باشند؛ لذا نیاز به بررسی و تأیید DBA یا ارشد تیم جهت جلوگیری از قفل شدن جدول‌ها دارد.

برای مدیریت ریسک و بازگشت سریع به حالت پایدار در صورت بروز خطا:

۱. بازگشت اپلیکیشن: از آنجا که تمام ایمیج‌ها بر اساس Commit SHA تگ خورده‌اند، در صورت شناسایی خطای بحرانی در Production، پایپلاین می‌تواند با یک کلیک یا به صورت خودکار، ترافیک را به تگ ایمیج نسخه قبلی هدایت کند.

۲. بازگشت پایگاه داده:

• اسکرپت‌های Migration باید همواره دارای متد Down باشند.

• تمام تغییرات دیتابیس باید بر اساس اصل Expand and Contract طراحی شوند تا نسخه جدید کدهای Backend با ساختار قبلی و نسخه جدید دیتابیس به‌طور هم‌زمان سازگار باشند.

پاسخ مسئله‌ی ۳. خرس

خواسته‌ی کارمی درست است: در تیمی که چند نفر هم‌زمان روی منو، سفارش و هماهنگی آشپزخانه کار می‌کنند، تنها جایی که می‌توان جلوی ورود مشکل به شاخه‌ی اصلی را گرفت، قبل از ادغام است. بعد از ادغام، هزینه‌ی پیدا کردن مقصر و برگرداندن تغییر چند برابر می‌شود.

تفاوت پایپلاین merge request با پایپلاین شاخه‌ی اصلی:

پایپلاین main	پایپلاین MR	
آیا آماده‌ی انتشار است؟	آیا اجازه‌ی ورود دارد؟	هدف
خود main	نتیجه‌ی ادغام شاخه با main	کد مورد آزمون
staging و سپس production	هیچ (یا حداکثر review app موقت)	استقرار
پوش ایمیج با تگ نسخه	فقط بیلد، بدون پوش پایدار	انتشار artifact
کل مجموعه، شامل e2e کامل	تست‌های سریع + تست‌های متأثر	دامنه‌ی تست
هشدار فوری به تیم، مسدودسازی انتشار	جلوی merge را می‌گیرد	سیاست شکست

مهم‌ترین تفاوت فنی، همان سطر دوم است. اجرای پایپلاین روی آخرین کامیت شاخه‌ی توسعه‌دهنده کافی نیست، چون ممکن است آن شاخه سبز باشد ولی بعد از ادغام با main خراب شود (semantic conflict: مثلاً یک نفر امضای تابع محاسبه‌ی قیمت سفارش را عوض کرده و نفر دیگر هم‌زمان یک فراخوانی جدید به همان تابع اضافه کرده است؛ گیت تعارض نمی‌بیند ولی کد کامپایل نمی‌شود). بنابراین پایپلاین باید روی merge result اجرا شود و اگر main جلو رفت، دوباره اجرا شود.

مراحل پایپلاین merge request:

۱. بررسی‌های ارزان: قالب پیام کامیت، نبود فایل حجیم، secret detection، لینت فقط روی فایل‌های تغییرکرده. این‌ها اول اجرا می‌شوند تا بازخورد تقریباً بلافاصله برسد.
۲. lint و بررسی تایپ: formatter، لینتر و کامپایل تایپ روی کل پروژه.
۳. unit test: سریع، بدون وابستگی خارجی، به همراه گزارش پوشش و مقایسه با main (کاهش پوشش، خودش یک هشدار است).
۴. build: بیلد ایمیج سرویس‌ها. اگر کد کامپایل شود ولی ایمیج ساخته نشود، مشکل واقعی است و باید همین‌جا دیده شود.
۵. integration test: با بالا آوردن وابستگی‌های واقعی (پایگاه داده، Redis، صف سفارش‌ها) به‌صورت سرویس کنار جاب؛ مسیرهای حساسی مثل ثبت سفارش و اعلان به آشپزخانه.
۶. بررسی امنیتی: SAST روی diff، اسکن وابستگی‌ها و اسکن ایمیج ساخته‌شده.
۷. review app: یک نمونه‌ی موقت از کل سامانه با URL یکتا برای هر MR، تا کارمی بتواند تغییر منو را واقعا ببیند نه اینکه تصورش کند. با بسته شدن MR خودکار حذف می‌شود. البته این مورد اختیاری هست و معمولاً استفاده نمی‌شود.

مسدودکننده (blocking):

- شکست کامپایل یا بیلد ایمیج
- حداقل یکی از تست‌های واحد یا یکپارچگی شکست‌خورده باشد
- خطای لینت و formatter

- آسیب‌پذیری high یا critical در وابستگی‌ها، و هر secret پیدا شده در کد
 - افت پوشش تست زیر آستانه‌ی توافق‌شده
 - غیرمسدودکننده (فقط هشدار، allow_failure):
 - آسیب‌پذیری low/medium بدون وصله‌ی موجود
 - هشدارهای کارایی مثل افزایش حجم bundle یا زمان بیلد
- معیار ساده: چیزی که قطعیت دارد و قابل رفع توسط نویسنده‌ی همین تغییر است، مسدودکننده باشد؛ چیزی که قضاوتی است یا رفعش به تصمیم تیم نیاز دارد، هشدار باشد.
- رساندن نتیجه به بازیبن‌ها و کارمی:
- وضعیت روی خود MR: هر جاب یک status check جداگانه با نام گویا؛ بازیبن در یک نگاه می‌فهمد کدام مرحله خراب است، نه اینکه فقط قرمز ببیند.
 - قوانین حفاظت از شاخه: دکمه‌ی merge تا سبز شدن بررسی‌های اجباری غیرفعال باشد. اتکا به یادت باشد قبل از merge نگاه کنی جواب نمی‌دهد.
 - گزارش‌های خلاصه به صورت کامنت خودکار: تست‌های شکست‌خورده با نام و پیام، اختلاف پوشش نسبت به main، و فهرست آسیب‌پذیری‌های تازه‌وارد شده همه در قالب diff، نه فهرست کامل؛ چون بازیبن فقط مسئول تغییر است.
 - artifact‌های قابل دانلود: گزارش پوشش، خروجی تست‌ها، لاگ کامل هر جاب و لاگ تست‌های e2e شکست‌خورده.
 - لینک review app مستقیم در توضیحات MR برای بررسی دستی توسط کارمی.
 - قوانین تأیید: حداقل یک تأیید از صاحب آن بخش (CODEOWNERS)؛ برای مثال تغییر در ماژول آشپزخانه نیاز به تأیید مسئول همان ماژول دارد.
- با این ساختار، تصمیم کارمی مبتنی بر شواهد است: پایپلاین می‌گوید چه چیزی خراب است، بازیبن می‌گوید آیا این راه‌حل درستی است، و دکمه‌ی merge تنها وقتی فعال می‌شود که هر دو پاسخ مثبت باشند.

پاسخ مسئله‌ی ۴. داکرایز

مراحل داکرایز کردن

مرحله اول: Builder Stage

- **FROM python:3.10-slim AS builder**: ایجاد یک مرحله مستقل با نام builder برای ایزوله کردن فرایند ساخت و کامپایل وابستگی‌ها.
- **ENV PYTHONDONTWRITEBYTECODE=1**: جلوگیری از تولید فایل‌های .pyc.
- **ENV PYTHONUNBUFFERED=1**: ارسال مستقیم خروجی‌های stdout و stderr به لاگ‌های داکر بدون buffer شدن.
- **ARG MIRROR_URL=""**: تعریف یک متغیر مخصوص زمان build برای تغییر منبع دانلود بسته‌ها.
- **RUN --mount=type=cache,target=/root/.cache/pip**: استفاده از قابلیت BuildKit Cache Mount برای کش کردن بسته‌های دانلودشده توسط pip و کاهش زمان بیلدهای بعدی.
- **-prefix=/install**: نصب وابستگی‌ها در مسیر ایزوله /install تا بتوان آن‌ها را به مرحله نهایی منتقل کرد.

مرحله دوم: Runner Stage (Final Image)

- **FROM python:3.10-slim AS runner**: شروع یک مرحله جدید و سبک بدون ابزارها و کش‌های مرحله ساخت.
- **PYTHONPATH و PATH**: افزودن مسیر بسته‌های نصب‌شده به متغیرهای محیطی.
- **RUN addgroup ... && adduser ...**: ایجاد یک کاربر Non-root برای افزایش امنیت و جلوگیری از حملات Container Breakout.
- **COPY --from=builder /install /install**: انتقال وابستگی‌های نصب‌شده بدون انتقال ابزارهای مرحله ساخت.
- **RUN chown -R appuser:appgroup /app**: واگذاری مالکیت فایل‌های پروژه به کاربر غیرریشه.
- **USER appuser**: اجرای برنامه با کاربر غیرریشه.

نحوه Build و اجرا

ساخت ایمیج با فعال بودن BuildKit:

```
1 DOCKER_BUILDKIT=1 docker build \
2   --build-arg MIRROR_URL=https://pypi.tuna.tsinghua.edu.cn/simple \
3   -t my-python-app:v1 .
```

اجرای کانتنر:

```
1 docker run --rm my-python-app:v1
```

نقش کلیدواژه‌ها

ENV متغیر محیطی تعریف می‌کند که هم در زمان build و هم در زمان اجرای کانتینر در دسترس است و در metadata ایمیج ذخیره می‌شود. بنابراین محل مناسبی برای نگهداری اطلاعات محرمانه نیست.

WORKDIR پوشه کاری دستورات بعدی مانند RUN، COPY و CMD را تعیین می‌کند و در صورت نیاز آن را ایجاد می‌کند.

AS به یک مرحله ساخت نام می‌دهد تا بتوان از آن در دستورهایی مانند COPY -from یا target- استفاده کرد.

ADD در برابر COPY

هر دو فایل‌ها را از build context به داخل ایمیج منتقل می‌کنند، اما ADD دو قابلیت اضافه دارد:

- استخراج خودکار آرشیوهای محلی؛
- دانلود فایل از یک URL (و در نسخه‌های جدید BuildKit، کلون کردن مخزن گیت).

به همین دلیل معمولاً استفاده از COPY توصیه می‌شود و فقط در صورت نیاز به استخراج خودکار آرشیو از ADD استفاده می‌شود.

EXPOSE در برابر Port-Mapping

EXPOSE 8000 فقط اعلام می‌کند که برنامه روی این پورت گوش می‌دهد و هیچ پورتنی را منتشر نمی‌کند.

در مقابل، docker run -p 8000:8000 یا کلید ports در Docker Compose باعث ایجاد نگاشت واقعی پورت روی میزبان می‌شود.

ARG در برابر ENV

ENV	ARG	
زمان build و زمان اجرا	فقط زمان build	دامنه حیات
env_file یا مقدار پیش فرض -e	-build-arg	تعیین مقدار
بله، در metadata	خیر (اما در docker history)	حضور در ایمیج نهایی
بله	خیر	دیده شدن در برنامه
به مراحل بعدی ارث می‌رسد	فقط همان مرحله	دامنه اثر

البته که هیچ‌کدام برای نگهداری secret مناسب نیستند.

CMD در برابر ENTRYPOINT

ENTRYPOINT مشخص می‌کند کانتینر چه فرایندی را اجرا می‌کند و معمولاً ثابت در نظر گرفته می‌شود؛ CMD آرگومان‌های پیش فرضی است که به آن فرایند داده می‌شود و کاربر می‌تواند هنگام docker run آن را بازنویسی کند. وقتی هر دو با هم بیایند، دستور نهایی از الحاق این دو ساخته می‌شود:

```
1 ENTRYPOINT ["uvicorn"]
2 CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `docker run img ← docker run img -host 0.0.0.0 -port 8000` (چیزی داده نشده، پس CMD پیش فرض به ENTRYPOINT الحاق می‌شود).
- `docker run img -version ← docker run img -version` (آرگومان داده شده فقط جای CMD را می‌گیرد، ENTRYPOINT ثابت می‌ماند).
- تغییر خود ENTRYPOINT فقط با فلگ `docker run -entrypoint ...` امکان پذیر است.

اگر فقط CMD تعریف شود (بدون ENTRYPOINT)، هر آرگومانی که در `docker run` داده شود کل دستور را جایگزین می‌کند؛ این رفتار برای ایمیج‌های چندمنظوره مناسب است، ولی برای یک سرویس مشخص، ترکیب ENTRYPOINT به همراه CMD رفتاری ثابت‌تر و قابل پیش‌بینی‌تر می‌دهد.

نکته مهم دیگر این‌که هر دو باید به صورت `exec` (آرایه JSON، مانند مثال بالا) نوشته شوند، نه به صورت `shell`. در قالب `shell` (مثلاً `python main.py` CMD)، فرایند اصلی برنامه زیر `/bin/sh` اجرا می‌شود و خودش PID 1 نیست؛ در نتیجه سیگنال `SIGTERM` که `docker stop` می‌فرستد به آن نمی‌رسد و کانتینر به جای خاموشی تمیز، تا رسیدن `timeout` معطل می‌ماند و سپس با `SIGKILL` بسته می‌شود.

ساخت و اجرای کانتینر از روی Dockerfile

```

1 # build
2 docker build -t my-python-app:v1 .
3
4 # run
5 docker run --rm my-python-app:v1
6
7 # verify
8 docker run --rm my-python-app:v1 whoami
9
10 # check size and layers
11 docker image inspect my-python-app:v1 --format '{{.Size}}'
12 docker history my-python-app:v1

```

از آن‌جا که این برنامه یک اسکریپت یک‌بارمصرف است، نه یک سرویس شنونده روی پورت، نیازی به `-p`، `EXPOSE` یا `docker compose` ندارد؛ کانتینر پس از اجرای `main.py` و چاپ خروجی به صورت خودکار خاتمه می‌یابد و به لطف `rm` حذف هم می‌شود.